Um sistema seu, com STL e IA

Atividade da Unidade 2 - planejar e construir com IA

Linguagem de Programação I

Sumário

Um sistema seu, com STL e IA **1**

1. A ideia . . . 1
2. O que o sistema precisa ter . . . 1
3. Ideias . . . 2
4. Como trabalhar . . . 2
5. Boas práticas para pedir código à IA . . . 3
6. Desafios opcionais . . . 5
7. Entrega . . . 5
8. Como será avaliado . . . 6
9. Uso responsável . . . 6

Instruções para a IA neste projeto **6**

Descrição do sistema **6**

Nome e ideia . . . 6
O que o sistema guarda . . . 7
O que dá para fazer . . . 7
Regras . . . 7
Quando algo dá errado . . . 7
Classes que eu imagino . . . 7
Exemplo de uso . . . 7

Um sistema seu, com STL e IA

Atividade da Unidade 2 de Linguagem de Programação I.

1. A ideia

Nesta atividade você vai construir um sistema em C++ que usa a STL, e vai construir com IA desde o começo. A diferença para os astronautas é que agora ninguém entrega o enunciado do sistema pronto: é você quem escolhe o tema, descreve o que o sistema faz e planeja com a IA como ele vai ser feito.

O foco é aprender a pedir. A mesma IA escreve código bom ou código ruim dependendo de como você pede, do que você deixa claro antes e do que você confere depois. A seção 5 reúne as práticas que eu quero ver no seu trabalho.

Antes de começar, faça os exercícios da aula de STL (exercícios 1 a 5 e o exercício final), sem IA. Eles não são entregues, mas são eles que vão fazer você entender o código que a IA escrever. Na conversa de avaliação eu vou perguntar sobre os containers que você usou.

O que você entrega é o link de um repositório público. Vou criar uma tarefa no SIGAA para você enviar o link. O prazo é o combinado com a turma e fica registrado nessa tarefa.

2. O que o sistema precisa ter

- Um tema escolhido por você. Pode ser uma das ideias da seção 3 ou outra.
- Pelo menos três classes suas, cada uma com uma responsabilidade clara.

- Uso da STL:
  - pelo menos um `map`;
  - pelo menos um `set` ou uma `list`;
  - `sort` ou `find` aplicado a objetos de uma classe sua, com `operator<` (ou `operator==`, no caso do `find`) definido nessa classe, como no exemplo 11 da aula.
- Interface em modo texto: um menu com opções numeradas ou comandos lidos da entrada padrão, você escolhe.
- A lógica separada da interface (a prática 5.5 explica o que isso quer dizer).
- C++11 e somente a biblioteca padrão.
- Compilar com o comando que estiver escrito no `README.md`.

Salvar os dados em arquivo não é obrigatório, porque a aula de arquivos ainda não aconteceu. Está entre os desafios da seção 6.

3. Ideias

Cada ideia traz uma descrição curta e classes que podem existir. São sugestões: você pode mudar nomes, juntar ou separar classes, e acrescentar funcionalidades. A descrição completa, com regras, é você quem escreve.

Estante de leituras. Registra livros lidos e por ler, com nota e comentário. Mostra os livros de um autor, os gêneros que você mais lê e os mais bem avaliados. Classes: `Livro`, `Autor`, `Estante`.

Campeonato. Cadastra times, registra resultados de partidas e mostra a tabela de classificação, com critério de desempate. Classes: `Time`, `Partida`, `Campeonato`.

Inventário de RPG. Um personagem coleta, usa e descarta itens, com limite de peso e itens que se acumulam (três poções ocupam uma entrada com quantidade 3). Classes: `Item`, `Personagem`, `Inventario`.

Playlist. Uma biblioteca de músicas, playlists com nome e uma fila de reprodução em que dá para adicionar no começo ou no fim, pular e voltar. Classes: `Musica`, `Playlist`, `Biblioteca`.

Agenda de consultas. Médicos com horários, pacientes que marcam e desmarcam consultas, e uma lista de espera para quando não há horário livre. Classes: `Medico`, `Paciente`, `Consulta`, `Agenda`.

Cartões de vocabulário. Cartões com palavra e tradução, estudados em sessões: o cartão que você erra volta para o fim da fila. Mostra as palavras que você mais erra. Classes: `Cartao`, `Baralho`, `Sessao`.

Controle de gastos. Registra gastos com valor, data e categoria. Mostra o total por categoria, os maiores gastos do mês e as categorias em que você passou do limite que definiu. Classes: `Gasto`, `Categoria`, `Carteira`.

4. Como trabalhar

4.1 A ferramenta

Use o OpenCode como na atividade dos astronautas (seção 4.1 daquele enunciado): `/connect` com OpenCode Zen, `/models` para escolher um modelo gratuito, `/new` para conversa nova, `/undo` para desfazer. O Big Pickle normalmente está gratuito e foi o que eu testei. Pode experimentar outros e anote no `README.md` qual usou.

Não execute `/init`: ele reescreve o `AGENTS.md`.

4.2 As etapas

1. Crie o repositório com o conteúdo desta pasta.
2. Escreva o `DESCRICAO.md` sem IA e faça um commit só com ele. É a descrição completa do sistema, e o modelo que está no arquivo diz o que ela precisa ter.
3. Peça à IA que aponte o que ficou ambíguo na descrição (prática 5.1). Responda, atualize o `DESCRICAO.md` e faça um commit.
4. Peça o plano em etapas, salvo em `PLANO.md` (prática 5.3). Leia o plano, discuta, corrija. Faça um commit.
5. Implemente uma etapa do plano por vez. Ao fim de cada etapa: compile, rode, confira, faça um commit.

6. Preencha a seção "Onde uso a STL" do `README.md`, sem IA.
7. Se sobrar tempo, escolha um desafio da seção 6.

Durante o trabalho é normal descobrir que a descrição tinha um buraco ou uma regra que não funciona. Quando isso acontecer, atualize o `DESCRICAO.md` e faça um commit. O histórico mostrando a descrição mudando é bom sinal.

5. Boas práticas para pedir código à IA

A IA preenche tudo o que você não disse com uma suposição dela. Quase todas as práticas abaixo servem para diminuir esse espaço, ou para você perceber rápido quando a suposição dela foi ruim.

5.1 Descreva antes de pedir

Pedir "faz um sistema de biblioteca" funciona, e o resultado é o sistema de biblioteca que a IA imaginou, não o seu. Por isso o `DESCRICAO.md` vem antes de qualquer código. Nos pedidos, aponte para ele em vez de reescrever:

```
Leia @DESCRICAO.md. Não altere nenhum arquivo. Me diga quais regras você
achou ambíguas ou incompletas e que perguntas você me faria antes de
programar.
```

Esse pedido costuma revelar coisas que você não tinha pensado, como o que acontece quando alguém tenta cadastrar um livro que já existe. A lista de perguntas pode vir longa. Responda as que mudam o funcionamento do sistema, escreva as respostas no `DESCRICAO.md` e faça um commit. Para as de detalhe, diga que ela pode escolher o jeito mais simples.

5.2 Coloque as regras permanentes no `AGENTS.md`

O `AGENTS.md` é lido pela IA antes de cada pedido. O que vale para o projeto inteiro fica lá: versão do C++, bibliotecas permitidas, organização dos arquivos, o que ela não deve fazer. Assim você não precisa repetir isso em toda mensagem. O arquivo já vem com regras; você pode acrescentar as suas. Se perceber que a IA insiste num erro, por exemplo usar um recurso que você não quer, escreva a regra lá.

5.3 Peça o plano antes do código

Nos astronautas você fez isso em dois tempos: plano primeiro, liberação depois. Aqui vale o mesmo, em escala maior. O primeiro plano é o do sistema inteiro:

```
Leia @DESCRICAO.md e @AGENTS.md. Não escreva código ainda. Proponha as
classes, o que cada uma guarda, quais containers da STL você usaria em cada
lugar e por quê, e divida a implementação em etapas pequenas, cada uma
terminando com algo que compila e que eu consigo testar. Salve em PLANO.md.
```

Leia o plano com atenção. É o momento mais barato para mudar de ideia. Se a IA escolheu um `vector` onde você acha que cabe um `map`, pergunte o motivo. Se ela propôs uma classe que você não entende para que serve, pergunte.

5.4 Um passo por vez

Pedir o sistema inteiro de uma vez gera muito código que você não leu, e quando algo dá errado fica difícil achar onde. Peça uma etapa do plano por mensagem, e só avance quando ela estiver funcionando e commitada:

```
Implemente somente a etapa 2 do @PLANO.md. Antes de editar, me diga quais
arquivos vai criar ou alterar.
```

Comece cada etapa com `/new`. O contexto que importa está nos arquivos (`DESCRICAO.md`, `PLANO.md`, `AGENTS.md`), não na conversa anterior. Conversa muito longa faz a IA se confundir com coisas que já mudaram.

5.5 Separe a lógica da interface

As classes do sistema (a `Estante`, o `Campeonato`) não usam `cin` nem `cout`. Elas recebem dados por parâmetro e devolvem resultados: um `bool` dizendo se deu certo, um número, um `vector` com o que foi pedido. Quem lê do teclado e escreve na tela é a interface, que fica em arquivos próprios.

Errado, a regra e a tela misturadas:

```
void Estante::avaliar(string titulo, int nota) {
    if (nota < 0 || nota > 10) {
        cout << "Nota invalida" << endl;
        return;
    }
    // ...
}
```

Certo, a classe só decide; a interface é quem escreve:

```
bool Estante::avaliar(string titulo, int nota) {
    if (nota < 0 || nota > 10) {
        return false;
    }
    // ...
    return true;
}
```

```
// no menu
if (!estante.avaliar(titulo, nota)) {
    cout << "Nota invalida" << endl;
}
```

Duas vantagens. Dá para testar a lógica sem digitar nada no menu. E dá para trocar a interface sem mexer na lógica: é isso que torna possível o desafio do modo gráfico. A IA mistura as duas coisas com frequência se você não pedir; a regra já está no `AGENTS.md`, mas confira no código.

5.6 Organize em arquivos

Uma classe por par de arquivos, `.hpp` e `.cpp`, como na aula de modularização. A interface em arquivos separados. Um comando de compilação que funciona e está escrito no `README.md`. Quando a IA criar um arquivo novo, confira se ele entrou no comando de compilação.

5.7 Diga o que fazer com o que dá errado

A IA escreve primeiro o caminho feliz: todo mundo digita certo, todo código existe, nenhuma lista está vazia. Os casos ruins só aparecem se você pedir. Na descrição e nos pedidos, diga o que deve acontecer quando:

- o código ou nome procurado não existe;
- o item cadastrado já existe;
- a lista está vazia;
- o usuário digita texto onde o programa esperava um número.

Este último derruba muitos menus em C++. Teste de propósito.

5.8 Diga antes qual é o resultado esperado

Antes de pedir uma funcionalidade, escreva um exemplo de uso e o que deve aparecer na tela. Se o seu programa lê da entrada padrão, esse exemplo vira um arquivo que você roda sempre:

```
./sistema < exemplos/cenario1.txt
```

Guarde esses arquivos em `exemplos/`. Depois de cada etapa, rode todos. É o jeito de saber que a etapa nova não quebrou a anterior.

5.9 Não aceite o que você não entende

Se a IA escreveu algo que você não sabe explicar, pergunte antes de seguir:

```
Explique a função Campeonato::classificacao linha por linha, como se eu
tivesse acabado de aprender map e sort. Por que map e não vector aqui?
```

Se a explicação usa coisas que você não viu, peça uma versão mais simples. Pedir lambda, `auto` complicado ou `unordered_map` a menos é um pedido legítimo. O código é seu, você vai responder por ele.

5.10 Confira você mesmo

A IA diz "compilei e funcionou" mesmo quando não compilou. Depois de cada etapa:

- compile você mesmo com o comando do `README.md`, com `-Wall -Wextra`, e leia os avisos;
- rode o programa e os arquivos de `exemplos/`;
- olhe `git status` e `git diff` e procure arquivos que você não esperava ver mudar;
- se não serve, `/undo` ou `git checkout -- .` e peça de novo, melhor. O `git checkout` não apaga arquivo novo que a IA criou; ele aparece no `git status` e você apaga à mão.

5.11 Controle o tamanho

Peça somente o que está na etapa. Se a IA acrescentar coisa que ninguém pediu (um menu de configurações, um sistema de login, cores no terminal), peça que tire. Código a mais é código que você também precisa entender e explicar.

6. Desafios opcionais

Só depois do sistema obrigatório funcionar e estar commitado. Cada desafio pede algo que as aulas ainda não mostraram, então é a IA que vai ensinar. A versão em modo texto continua obrigatória e precisa continuar compilando só com a biblioteca padrão.

Os desafios 2 e 3 usam uma biblioteca de fora. Quando começar um deles, atualize o `AGENTS.md` dizendo que aquela biblioteca passou a ser permitida, e só no código daquele desafio.

1. Salvar e carregar em arquivo texto. Os dados sobrevivem ao fechar o programa. Peça que a leitura e a escrita fiquem numa classe própria, fora das classes do sistema. É o desafio mais simples dos três.

2. Banco de dados SQLite. Em vez de arquivo texto, os dados ficam num banco SQLite, usando a biblioteca `sqlite3` em C. Precisa instalar a biblioteca (no macOS já vem; no Linux, `libsqlite3-dev`; no Windows é mais trabalhoso) e compilar com `-lsqlite3`. Peça que o acesso ao banco fique numa classe própria e que o `.db` não entre no repositório.

3. Interface gráfica. Uma janela no lugar do menu, usando a raylib, a mesma que eu mostrei em sala. Precisa instalar a biblioteca. Aqui a prática 5.5 faz toda a diferença: as classes do sistema continuam as mesmas, e só a interface é nova. O ideal é ter dois executáveis, um em modo texto e outro gráfico, compilados a partir das mesmas classes. Instalação e interface gráfica tomam tempo; não comece por aqui.

Se escolher um desafio, escreva no `README.md` como instalar o que for preciso e como compilar a versão do desafio.

7. Entrega

Envie na tarefa do SIGAA o link do seu repositório público. Ele precisa ter:

- `DESCRICAO.md` escrito por você, com um commit anterior a qualquer código;
- `PLANO.md`;
- o código compilando com o comando do `README.md`;
- os arquivos de `exemplos/`;
- o `README.md` preenchido, inclusive "Onde uso a STL" e "Como usei a IA";
- um commit por etapa, mais ou menos;
- nenhum executável, arquivo `.o` ou `.db` commitado.

8. Como será avaliado

- A descrição: está completa, veio antes do código, e o sistema faz o que ela diz.
- Os requisitos da seção 2, inclusive os de STL.
- A organização: lógica separada da interface, uma classe por arquivo, compila sem erros.
- A seção "Onde uso a STL", com as suas palavras, batendo com o código.
- O histórico: etapas pequenas, uma por commit, em vez de um único commit com tudo.
- Conversa de cerca de cinco minutos comigo, sobre as escolhas de container e sobre uma parte do código que a IA escreveu.

Os desafios contam a favor de quem fizer, mas não substituem nada do que é obrigatório.

9. Uso responsável

- Use só dados fictícios.
- Não cole chaves, senhas ou tokens em pedido, código ou README.
- Modelos gratuitos podem guardar as conversas. Não envie nada que não possa ser público.
- O código entregue é seu. Você responde por ele, inclusive pelo que a IA escreveu.

Instruções para a IA neste projeto

- Este é o projeto de um estudante de C++ que está aprendendo STL. Responda em português, de forma curta e didática. Quando explicar um conceito, use um exemplo pequeno.
- O sistema está descrito em `DESCRICAO.md`, escrito pelo estudante. O plano de implementação fica em `PLANO.md`. Leia os dois antes de propor ou alterar código. Se a descrição for ambígua, pergunte em vez de supor.
- Use C++11 e somente a biblioteca padrão. Não instale nada e não acesse a rede.
- Prefira guardar objetos diretamente nos containers da STL. Não use `new` nem `delete`, a não ser que o estudante peça herança com método virtual; nesse caso, explique quem libera a memória.
- Evite recursos que o estudante ainda não viu, como lambda e `unordered_map`, a não ser que ele peça. Se achar que um deles ajuda, sugira e explique antes de usar.
- Uma classe por par de arquivos: `include/Nome.hpp` e `src/Nome.cpp`.
- As classes do sistema não usam `cin` nem `cout`. Elas recebem dados por parâmetro e devolvem resultados. Toda leitura e escrita na tela fica na interface, em arquivos próprios.
- Faça somente o que foi pedido nesta mensagem, ou a etapa do `PLANO.md` que o estudante indicou. Não acrescente funcionalidades que não foram pedidas.
- Quando o estudante pedir um plano, não edite: diga quais arquivos e quais métodos você criaria ou alteraria, e por quê.
- Depois de editar, compile com o comando que está no `README.md` e rode os arquivos de `exemplos/`, se existirem. Mostre o resultado real. Se criar um arquivo novo, atualize o comando de compilação no `README.md`.
- Não altere `DESCRICAO.md`, nem as seções "Onde uso a STL" e "Como usei a IA" do `README.md`. Essas partes são escritas pelo estudante. Se achar que a descrição precisa mudar, sugira a mudança.
- Não faça commit e não mexa no histórico do Git.

Descrição do sistema

Escreva este arquivo sem IA, antes de qualquer código. Apague as instruções em itálico quando preencher.

Nome e ideia

Duas ou três frases: o que o sistema é e para quem serve.

O que o sistema guarda

As coisas que o sistema cadastra (livros, times, itens…) e o que cada uma tem (nome, preço, quantidade…).

O que dá para fazer

Cada opção do menu ou cada comando, com o que ele pede e o que mostra.

Regras

O que é permitido e o que não é. Por exemplo: não pode haver dois livros com o mesmo título; nota vai de 0 a 10; a tabela ordena por pontos e, empatando, por saldo de gols.

Quando algo dá errado

O que o sistema faz quando o item procurado não existe, quando o cadastro já existe, quando a lista está vazia, quando a entrada é inválida.

Classes que eu imagino

Uma linha para cada classe: o nome e a responsabilidade dela. Não precisa ser a versão final; o plano com a IA pode mudar isso.

Exemplo de uso

Uma sequência curta de uso e o que aparece na tela, do jeito que você imagina.